
SECURE DEVSECOPS INFRASTRUCTURE

Design Report

Architecture Diagrams • Deployment Workflows
Identity & Access Management • GitOps Workflows

Name	T G JAI KOUSICK
Register Number	192421367
Subject	DevSecOps Engineering and Application Security
Subject Code	CSA5802
Department	B.Tech Information Technology

1. Secure DevSecOps Architecture Overview

A secure DevSecOps architecture integrates security controls into every stage of the software development lifecycle (plan → code → build → test → release → deploy → operate → monitor) while maintaining velocity through automation. Security is treated as a shared responsibility and is automated wherever possible.

High-level Reference Architecture (Logical Layers)

- **Source Control & Collaboration Layer:** Git repository (GitHub / GitLab / Bitbucket) with branch protection, signed commits, CODEOWNERS, and required status checks.
- **CI/CD Pipeline Layer:** Automated pipelines (GitHub Actions, GitLab CI, Jenkins, Azure DevOps, or Tekton) that enforce security gates as quality gates.
- **Security Tooling Layer:** SAST, SCA, DAST, IaC scanning, container scanning, secret scanning, and policy-as-code (OPA/Gatekeeper, Kyverno).
- **Artifact & Registry Layer:** Signed, immutable artifacts in a private container registry / artifact repository (Harbor, ECR, ACR, Artifactory) with vulnerability scanning and admission control.
- **Runtime / Platform Layer:** Kubernetes (or equivalent) with network policies, Pod Security Standards / Admission Controllers, service mesh (Istio/Linkerd) for mTLS, and runtime threat detection (Falco, Aqua, Sysdig).
- **Identity & Access Layer:** Centralized IAM (OIDC/OAuth2 + RBAC + least privilege) integrated with cloud IAM and Kubernetes RBAC.
- **Observability & Feedback Layer:** Centralized logging, metrics, tracing, security events (SIEM), and continuous compliance dashboards.
- **GitOps Control Plane:** Declarative desired-state repositories synchronized by operators (Argo CD / Flux).

Key Security Principles Embedded

- Shift-left security — find and fix issues as early as possible in the lifecycle.
- Defense-in-depth — multiple independent layers of control.
- Least privilege and zero-trust access models.
- Immutable infrastructure and artifacts.
- Policy-as-code with automated enforcement and continuous verification.
- Supply-chain security and attestation (SLSA, in-toto, Sigstore/Cosign).

2. Architecture Diagram (Logical Representation)

The following textual architecture illustrates the end-to-end secure flow from developer commit through GitOps-driven deployment and continuous runtime protection.

```
[ Developers / PR Reviewers ]
|
v
[ Git Repository ] ---- (signed commits, protected branches, required checks)
|
+---> [ CI Pipeline ]
|       |
|       +-- SAST / SCA / Secret Scan / IaC Scan / Unit Tests
|       +-- Build + Unit / Integration Tests
|       +-- Container Build + Image Scan
|       +-- Sign Artifact (Cosign) + SBOM generation
|       +-- Push to Secure Registry (only on success)
|
v
[ GitOps Repository (desired state) ] <--- (PR-based changes only)
|
v
[ GitOps Operator (Argo CD / Flux) ] ---- (pull-based reconciliation)
|
+---> [ Kubernetes Clusters ]
```

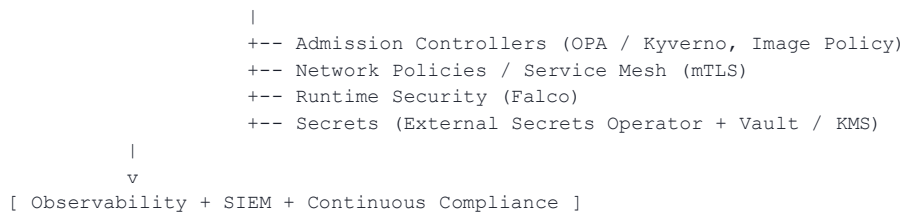


Figure 1: Secure DevSecOps Logical Architecture

Multi-environment view: Development → Staging → Production clusters (or strongly isolated namespaces). Separate GitOps Applications / Projects per environment. Production requires additional approvals, canary or blue-green progressive delivery, and stricter admission policies.

3. Modelling Deployment Workflows

Secure CI/CD + GitOps Deployment Workflow

- 1. Developer commits code** → feature branch → Pull / Merge Request created.
- 2. Automated checks on PR:** Linting, unit tests, SAST (SonarQube, CodeQL, Semgrep), SCA / dependency scanning (Snyk, Dependabot, Trivy), secret scanning (TruffleHog, Gitleaks), and IaC scanning (Checkov, tfsec, Terrascan) when infrastructure changes are present.
- 3. On merge to protected branch:** Full pipeline executes. Build immutable artifact (container image). Generate SBOM (Syft / CycloneDX). Scan image (Trivy, Clair, Aqua). Sign image with Cosign (keyless or key-based) and attach attestations. Push only signed, scanned images to the private registry.
- 4. Update desired state (GitOps repo):** Automated or manual PR updates image tag / Helm values / Kustomize overlays. The same security gates apply to changes in the GitOps repository.
- 5. GitOps reconciliation:** Argo CD / Flux detects drift and applies the desired state (pull model preferred for reduced attack surface). Admission webhooks enforce policies (no unsigned images, resource limits, allowed registries only, network rules).
- 6. Progressive delivery:** Canary / blue-green / progressive rollout with automated analysis (Argo Rollouts, Flagger). Automatic rollback on functional failure or security anomaly.
- 7. Runtime continuous verification:** Runtime scanning, anomaly detection, policy compliance checks. Feedback loop into backlog and pipeline improvements.

Mandatory Security Gates (Must-Pass)

- No critical or high vulnerabilities above defined thresholds (or documented time-bound exceptions).
- All artifacts cryptographically signed and verified at admission time.
- Secrets never stored in Git; use Sealed Secrets, External Secrets Operator, or Vault.
- Infrastructure-as-Code changes reviewed, scanned, and approved via Pull Request.
- Branch protection and required status checks enforced on both application and GitOps repositories.

4. Identity and Access Management (IAM) Relationships

Core principles: least privilege, just-in-time (JIT) access, separation of duties, complete auditability, and zero-trust. Human and machine identities are treated distinctly; long-lived credentials are avoided in favour of short-lived tokens and OIDC federation.

Key IAM Relationships and Role Mappings

Actor / Component	Identity Provider	Permissions / Role Model	Notes
Developers	Corporate IdP (OIDC/SAML) + Git provider	Read/write on feature branches; PR creation; limited pipeline trigger	No direct production access
Code Reviewers / Maintainers	Same IdP	Approve PRs; merge to protected branches	CODEOWNERS enforced

Actor / Component	Identity Provider	Permissions / Role Model	Notes
CI/CD Service Accounts	OIDC federation (preferred) or short-lived tokens	Push to registry (scoped); update specific GitOps paths; read secrets from Vault	No long-lived static keys
GitOps Operator (Argo CD / Flux)	Kubernetes ServiceAccount + OIDC	Sync only declared Applications; read-only on source repositories	Project-scoped RBAC
Kubernetes Workloads	Workload Identity / IRSA / Pod Identity	Cloud permissions via federated identity	Least-privilege IAM roles; no static keys
Platform / SRE Team	IdP + break-glass accounts	Cluster-admin (time-bound, audited); GitOps admin	JIT + MFA mandatory
Security / Compliance Team	IdP	Read-only dashboards, policy definition, exception approval	Separation of duties
End Users / Applications	Application IdP (OIDC)	Fine-grained application RBAC	Independent of platform IAM

Table 1: IAM Actor-to-Permission Mapping

Implementation Recommendations

- Prefer OIDC federation everywhere (GitHub Actions → cloud providers, Kubernetes Workload Identity).
- Use a central policy engine (OPA) for both pipeline decisions and cluster admission control.
- Ship all access and change logs to a SIEM; enable alerting on anomalous privileged actions.
- Conduct regular access reviews and automated de-provisioning on role or employment changes.
- Manage secrets via HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault together with the External Secrets Operator.
- Enforce MFA and hardware-backed keys (or WebAuthn) for all privileged human access.

5. GitOps Workflows

GitOps treats Git as the single source of truth for desired cluster state. Changes are versioned, immutable, and auditable. Pull-based reconciliation (Argo CD / Flux) is preferred over push-based models because it reduces the attack surface of the control plane.

Core Secure GitOps Principles

- Declarative desired state stored exclusively in Git.
- All changes performed via Pull Requests with mandatory review and security scanning.
- Automated drift detection and continuous reconciliation.
- Human operators never apply kubectl apply directly in production; the operator is the only writer.
- Environment promotion is itself a Git change (promotion PR or automated image-tag update).

Recommended Repository Structure

```

gitops-repo/
├── apps/
│   ├── base/           # common resources & shared config
│   ├── overlays/
│   │   ├── dev/
│   │   ├── staging/
│   │   └── prod/
│   ├── infrastructure/ # cluster add-ons, operators, policies
│   ├── policies/       # OPA / Kyverno policies as code
│   ├── CODEOWNERS
│   └── README.md

```

Figure 2: Typical Secure GitOps Repository Layout

Secure GitOps Operational Flow

- Application CI pipeline produces a signed image and opens (or updates) a PR in the GitOps repository that changes the image tag or values.

- The GitOps PR is subject to the same security scanning (IaC, policy validation, secret detection).
- After approval and merge, Argo CD / Flux reconciles the cluster to the new desired state.
- ApplicationSets or multi-source Applications manage multi-environment promotion cleanly.
- Sync waves and resource hooks enforce ordered deployment (e.g., database migrations before application pods).
- Health checks and automated analysis drive progressive delivery; failed syncs or health degradations trigger rollback.
- Every reconciliation and every human Git action is fully auditable in Git history and operator logs.

Additional Security Controls for GitOps

- Argo CD / Flux run under least-privilege Kubernetes ServiceAccounts and are restricted by NetworkPolicy.
- Repository credentials are injected via Sealed Secrets or an external secret store — never stored in clear text.
- Image Updater (if used) is limited to non-production environments or tightly allow-listed images.
- Progressive delivery tools (Argo Rollouts / Flagger) are integrated so that promotion itself is declarative.
- Audit logs of every sync operation are forwarded to the central SIEM.

6. Summary of Secure Controls Across the Lifecycle

Lifecycle Stage	Primary Controls
Plan / Code	Threat modelling, secure coding standards, pre-commit hooks, branch protection
Build	SAST, SCA, secret scanning, SBOM generation, artifact signing (Cosign)
Test	DAST, integration tests, policy validation, container image scanning
Release / Deploy	GitOps pull reconciliation, admission control, progressive delivery, signed images only
Operate	Runtime protection (Falco), network segmentation, continuous compliance, secrets rotation
Monitor	Observability stack, SIEM correlation, automated response playbooks, feedback into backlog

Table 2: Security Controls Mapped to SDLC Stages

This architecture provides a practical, defence-in-depth DevSecOps foundation suitable for cloud-native (Kubernetes) environments while remaining adaptable to hybrid and multi-cloud setups. It emphasises automation, verifiable supply-chain integrity, and continuous feedback so that security improves with every delivery cycle.

References (Conceptual Frameworks)

- CNCF GitOps Principles and GitOps Working Group guidance.
- SLSA (Supply-chain Levels for Software Artifacts) framework.
- NIST SP 800-204 series (Security Strategies for Microservices / Cloud-native).
- CIS Kubernetes Benchmarks and CIS Software Supply Chain Security Guide.
- Sigstore / Cosign, Open Policy Agent (OPA), Argo CD, Flux, Trivy, Falco (vendor-neutral tooling examples).